README

Exercise 2 Gesture Classification Input Sketch

Real-Time Gesture Classification Using Webcam Input,
Image Cropping, Downsampling, Pixel Feature Extraction,
OSC Communication, and Wekinator

`GestureInput.pde`

Contents

1 Purpose of This Report **4**

2 Big Picture Explanation **4**

3 What This Sketch Does **4**

4 Input and Output of the Sketch **5**
4.1 Input 5
4.2 Output 5

5 Relationship with Wekinator **6**

6 Important Concept: What Is a Feature Vector? **6**

7 Why Cropping Is Important **7**

8 Why Downsampling Is Important **7**

9 Why Grayscale Is Used **8**

10 Feature Configuration Used in This Sketch **8**

11 Possible Feature Configurations for the Lab **8**

12 Full Code Being Explained **9**

13 Line-by-Line and Block-by-Block Explanation **14**
13.1 Library Imports 14
13.2 Camera and OSC Objects 14
13.3 Wekinator OSC Settings 14
13.4 Local OSC Port 15
13.5 Feature Configuration 15
13.6 Image Variables 15
13.7 Preview and Sending Controls 15

14 The `setup()` Function **16**
14.1 OSC Setup 16
14.2 Camera Setup 16
14.3 Feature Count Calculation 17
14.4 Console Information 18
14.5 Font Setup 18

15 The `captureEvent()` Function **19**

16 The `draw()` Function **19**
16.1 Crop the Camera Frame 20
16.2 Downsample the Crop 20
16.3 Send Features to Wekinator 20
16.4 Draw the Interface 20

17 The `getCenterCrop()` Function **21**

18 The `sendImageFeaturesToWekinator()` Function **22**
18.1 RGB Mode . . . 22
18.2 Grayscale Mode . . . 23
18.3 End of Pixel Loop and Send Message . . . 23

19 The `drawCameraPreview()` Function **23**
19.1 Draw the Green Crop Rectangle . . . 24

20 The `drawCropPreview()` Function **25**

21 The `drawFeaturePreview()` Function **26**
21.1 Draw RGB Feature Preview . . . 26
21.2 Draw Grayscale Feature Preview . . . 26
21.3 Label the Feature Preview . . . 27

22 The `drawHUD()` Function **28**
22.1 OSC and Feature Information . . . 28
22.2 Sending Status . . . 29

23 The `keyPressed()` Function **30**

24 Complete Program Flow **30**

25 Wekinator Training Procedure with This Sketch **31**

26 Important Training Advice **32**

27 Why KNN Works with This Input **32**

28 How This Code Fulfils the Exercise 2 Requirements **33**

29 Strengths of This Sketch **34**
29.1 Clear Pipeline . . . 34
29.2 Easy Feature Configuration . . . 34
29.3 Good First Configuration . . . 34
29.4 Real-Time Visual Feedback . . . 34
29.5 Spacebar Control . . . 35
29.6 Wekinator Compatibility . . . 35

30 Limitations of This Sketch **35**
30.1 Center Crop May Not Always Capture the Hand . . . 35
30.2 Background Can Affect Classification . . . 35
30.3 Grayscale Removes Color Information . . . 35
30.4 Pixel Features Are High-Dimensional . . . 35

31 Recommended Experimental Results Discussion **36**
31.1 Resolution Comparison . . . 36
31.2 Grayscale vs RGB Comparison . . . 36

32 Debugging Checklist **36**

33 Final Explanation in Simple Words **37**

34 Final Conclusion **37**

1. Purpose of This Report

This README explains the `GestureInput.pde` Processing sketch completely and comprehensively.

The report has three main goals:

1. Explain the **Processing code itself** as a standalone program.
2. Explain the **functionality of each important line and block** of code.
3. Explain how this sketch fulfils the requirements of **Exercise 2 — Gesture Classification**.

This sketch is the **input side** of the gesture classification system. It does not perform machine learning by itself. Instead, it prepares visual data from the webcam and sends that data to Wekinator, where the classifier is trained.

2. Big Picture Explanation

The sketch captures live video from a webcam, extracts a square region from the center of the image, reduces that image to a smaller resolution, converts the pixels into numbers, and sends those numbers to Wekinator using OSC.

The purpose is to transform a human gesture, such as an open hand or a closed fist, into a numerical feature vector that a machine learning model can understand.

The complete idea is:

```
User shows a gesture to the webcam
        |
Processing captures the camera frame
        |
Processing crops a 200x200 region
        |
Processing resizes the crop to 50x50
        |
Processing converts pixels into grayscale numbers
        |
Processing sends 2500 numbers to Wekinator
        |
Wekinator classifies the gesture
```

This sketch is only the first half of the system. The second half is Wekinator and an output Processing sketch that receives the classification result.

3. What This Sketch Does

The sketch performs these tasks:

1. Imports the Processing Video library.
2. Imports the oscP5 and netP5 libraries for OSC communication.

3. Starts the webcam.
4. Creates an OSC connection to Wekinator.
5. Captures live camera frames.
6. Crops a 200×200 square region from the center of the frame.
7. Downsamples the crop to a smaller image, currently 50×50.
8. Converts the downsampled image into grayscale brightness values.
9. Sends those values to Wekinator as an OSC message.
10. Displays the camera, crop preview, feature image preview, and technical information on screen.
11. Allows the user to pause or resume sending data using the spacebar.

4. Input and Output of the Sketch

4.1. Input

The input is live video from the webcam.

The user places a gesture inside the crop area, such as:

- Open hand
- Closed fist
- One finger
- Two fingers
- Thumb up
- Palm facing camera

The webcam image is the raw input, but Wekinator does not receive the full image. Wekinator only receives the processed feature vector.

4.2. Output

The output of this sketch is an OSC message sent to Wekinator.

The message is sent to:

```
Host: 127.0.0.1
Port: 6448
OSC address: /wek/inputs
```

The current configuration sends:

```
2500 input values
```

because:

```
50 x 50 grayscale = 2500 features
```

Each feature is a normalized grayscale brightness value between `0.0` and `1.0`.

5. Relationship with Wekinator

Wekinator is the machine learning program. It receives the numerical feature vector and learns to associate those numbers with gesture classes.

For this sketch, Wekinator should be configured as follows:

Setting	Correct Value
Input port	`6448`
Input OSC message	`/wek/inputs`
Number of inputs	`2500`
Number of outputs	`1`
Output type	Classification
Number of classes	`2`
Output OSC message	`/wek/outputs`
Output host	`localhost` or `127.0.0.1`
Output port	`12000`

For example:

```
Class 1 -> Open Hand
Class 2 -> Closed Fist
```

6. Important Concept: What Is a Feature Vector?

A machine learning classifier cannot directly understand an image the way a human does.

A human sees:

```
Open hand
```

But Wekinator receives something like:

```
0.12, 0.15, 0.18, 0.20, ..., 0.76
```

These numbers are called a **feature vector**.

In this sketch, the feature vector is created from a 50×50 grayscale image.

Since the image has:

```
50 rows x 50 columns = 2500 pixels
```

and each pixel becomes one brightness number, the final feature vector contains:

```
2500 values
```

Each value represents how bright one pixel is:

- `0.0` means black or very dark.
- `1.0` means white or very bright.
- Values between `0.0` and `1.0` represent different gray levels.

7. Why Cropping Is Important

The sketch uses a 200×200 crop because Wekinator should focus on the important part of the image.

If the full webcam image is used, the classifier receives many irrelevant pixels:

- Background
- Wall
- Face
- Clothes
- Lighting changes
- Unrelated objects

Cropping reduces the input area and helps the classifier focus on the gesture.

However, the crop must contain the hand. If the green crop box contains mostly the face instead of the hand, Wekinator will learn the face/background instead of the gesture.

Correct training requires:

```
The hand gesture must be inside the green crop box.
```

8. Why Downsampling Is Important

The original crop is 200×200 pixels.

If we send the full grayscale crop to Wekinator, the number of inputs is:

```
200 x 200 = 40,000 inputs
```

This is very large and can make Wekinator harder to train.

The sketch downsamples the image to 50×50:

```
50 x 50 = 2500 inputs
```

This is much lighter and easier to handle.

Downsampling keeps the general shape of the gesture while reducing detail.

9. Why Grayscale Is Used

The current sketch uses grayscale mode:

```
boolean useRGB = false;
```

This means each pixel becomes one value: brightness.

Grayscale is useful because gesture classification often depends more on shape than color.

For open hand vs closed fist, the important information is usually:

- Shape of the hand
- Position of fingers
- Overall silhouette
- Brightness pattern

Using RGB would send three values per pixel:

```
Red, green, blue
```

For a 50×50 image, RGB mode would send:

```
50 x 50 x 3 = 7500 inputs
```

RGB can preserve color information, but it is heavier and more sensitive to lighting.

10. Feature Configuration Used in This Sketch

The current configuration is:

```
int cropSize = 200;
int featureSize = 50;
boolean useRGB = false;
```

This means:

Parameter	Meaning	Current Value
`cropSize`	Size of the original crop	200×200
`featureSize`	Size after resizing	50×50
`useRGB`	Whether to send RGB or grayscale	`false`, grayscale
`featureCount`	Number of values sent to Wekinator	2500

11. Possible Feature Configurations for the Lab

Exercise 2 asks students to experiment with different resolutions and color modes.

This sketch supports those experiments by changing only two variables:

```
int featureSize = 50;
boolean useRGB = false;
```

The possible configurations are:

Resolution	Color Mode	Number of Wekinator Inputs
200×200	Grayscale	40,000
200×200	RGB	120,000
100×100	Grayscale	10,000
100×100	RGB	30,000
50×50	Grayscale	2,500
50×50	RGB	7,500

The current version uses the simplest and most stable configuration:

```
50x50 grayscale = 2500 inputs
```

12. Full Code Being Explained

```
import processing.video.*;
import oscP5.*;
import netP5.*;

Capture cam;
OscP5 oscP5;
NetAddress wekinator;

String wekHost = "127.0.0.1";
int wekPort = 6448;
String oscAddress = "/wek/inputs";

int localPort = 12001;

int cropSize = 200;
int featureSize = 50;
boolean useRGB = false;
int featureCount;

PImage cropImg;
PImage featureImg;

boolean mirrorPreview = true;
boolean sendToWekinator = true;

void setup() {
  size(960, 720);
```

```
  oscP5 = new OscP5(this, localPort);
  wekinator = new NetAddress(wekHost, wekPort);

  String[] cameras = Capture.list();

  if (cameras == null || cameras.length == 0) {
    println("No camera found.");
    exit();
  }

  println("Available cameras:");
  for (int i = 0; i < cameras.length; i++) {
    println(i + ": " + cameras[i]);
  }

  cam = new Capture(this, cameras[0]);
  cam.start();

  if (useRGB) {
    featureCount = featureSize * featureSize * 3;
  } else {
    featureCount = featureSize * featureSize;
  }

  println("=====================================");
  println("Exercise 2 Input Sketch Started");
  println("Crop size: " + cropSize + " x " + cropSize);
  println("Feature image size: " + featureSize + " x " +
     featureSize);
  println("Color mode: " + (useRGB ? "RGB" : "Grayscale"));
  println("Number of Wekinator inputs: " + featureCount);
  println("OSC address: " + oscAddress);
  println("Sending to: " + wekHost + ":" + wekPort);
  println("=====================================");

  textFont(createFont("Arial", 16));
}

void captureEvent(Capture c) {
  c.read();
}

void draw() {
  background(0);

  if (cam.width == 0 || cam.height == 0) {
    return;
  }

  cam.loadPixels();
```

```
  cropImg = getCenterCrop(cam, cropSize);

  featureImg = cropImg.copy();
  featureImg.resize(featureSize, featureSize);
  featureImg.loadPixels();

  if (sendToWekinator) {
    sendImageFeaturesToWekinator(featureImg);
  }

  drawCameraPreview();
  drawCropPreview();
  drawFeaturePreview();
  drawHUD();
}

PImage getCenterCrop(PImage source, int size) {
  int startX = (source.width - size) / 2;
  int startY = (source.height - size) / 2;

  startX = constrain(startX, 0, source.width - size);
  startY = constrain(startY, 0, source.height - size);

  return source.get(startX, startY, size, size);
}

void sendImageFeaturesToWekinator(PImage img) {
  OscMessage msg = new OscMessage(oscAddress);

  img.loadPixels();

  for (int i = 0; i < img.pixels.length; i++) {
    color c = img.pixels[i];

    if (useRGB) {
      msg.add(red(c) / 255.0);
      msg.add(green(c) / 255.0);
      msg.add(blue(c) / 255.0);
    } else {
      float gray = brightness(c) / 255.0;
      msg.add(gray);
    }
  }

  oscP5.send(msg, wekinator);
}

void drawCameraPreview() {
  pushMatrix();

  if (mirrorPreview) {
```

```
    translate(width, 0);
    scale(-1, 1);
  }

  image(cam, 0, 0, width, height);

  popMatrix();

  noFill();
  stroke(0, 255, 0);
  strokeWeight(3);

  int rectX = (width - cropSize) / 2;
  int rectY = (height - cropSize) / 2;

  rect(rectX, rectY, cropSize, cropSize);
}

void drawCropPreview() {
  int previewSize = 160;
  int x = width - previewSize - 20;
  int y = 20;

  fill(0, 180);
  noStroke();
  rect(x - 10, y - 10, previewSize + 20, previewSize + 45);

  image(cropImg, x, y, previewSize, previewSize);

  fill(255);
  textSize(14);
  text("Center crop: 200 x 200", x, y + previewSize + 25);
}

void drawFeaturePreview() {
  int previewSize = 160;
  int x = width - previewSize - 20;
  int y = 240;

  fill(0, 180);
  noStroke();
  rect(x - 10, y - 10, previewSize + 20, previewSize + 65);

  if (useRGB) {
    image(featureImg, x, y, previewSize, previewSize);
  } else {
    PImage grayPreview = createImage(featureImg.width,
       featureImg.height, RGB);
    featureImg.loadPixels();
    grayPreview.loadPixels();
```

```
    for (int i = 0; i < featureImg.pixels.length; i++) {
      float gray = brightness(featureImg.pixels[i]);
      grayPreview.pixels[i] = color(gray);
    }

    grayPreview.updatePixels();
    image(grayPreview, x, y, previewSize, previewSize);
  }

  fill(255);
  textSize(14);
  text("Feature image: " + featureSize + " x " + featureSize,
     x, y + previewSize + 25);
  text("Mode: " + (useRGB ? "RGB" : "Grayscale"), x, y +
     previewSize + 45);
}

void drawHUD() {
  fill(0, 190);
  noStroke();
  rect(20, 20, 390, 170);

  fill(255);
  textSize(16);
  text("Exercise 2 -- Gesture Input Sketch", 35, 50);

  textSize(14);
  text("OSC address: " + oscAddress, 35, 80);
  text("Sending to Wekinator: " + wekHost + ":" + wekPort, 35,
     105);
  text("Crop: " + cropSize + " x " + cropSize, 35, 130);
  text("Feature size: " + featureSize + " x " + featureSize,
     35, 155);
  text("Color mode: " + (useRGB ? "RGB" : "Grayscale"), 35,
     180);
  text("Wekinator inputs: " + featureCount, 35, 205);

  fill(sendToWekinator ? color(0, 255, 0) : color(255, 0, 0));
  text("Sending: " + (sendToWekinator ? "ON" : "OFF"), 35, 230);

  fill(255);
  text("Press SPACE to start/stop sending", 35, height - 35);
}

void keyPressed() {
  if (key == ' ') {
    sendToWekinator = !sendToWekinator;
    println("Sending to Wekinator: " + sendToWekinator);
  }
}
```

13. Line-by-Line and Block-by-Block Explanation

13.1. Library Imports

```
import processing.video.*;
```

This imports the Processing Video library. It allows the sketch to use the webcam through the `Capture` class.

```
import oscP5.*;
```

This imports the oscP5 library. It allows the sketch to create and send OSC messages.

```
import netP5.*;
```

This imports netP5. It is used together with oscP5 to define network destinations such as Wekinator's IP address and port.

13.2. Camera and OSC Objects

```
Capture cam;
```

This declares a webcam object called `cam`. It will later store the live camera stream.

```
OscP5 oscP5;
```

This declares the main OSC communication object. It is used to send messages from Processing to Wekinator.

```
NetAddress wekinator;
```

This declares a network address object. It stores where Wekinator is listening, specifically the host and port.

13.3. Wekinator OSC Settings

```
String wekHost = "127.0.0.1";
```

This sets the Wekinator host address. `127.0.0.1` means the local computer, also called localhost.

```
int wekPort = 6448;
```

This sets the destination port for sending input features to Wekinator. Wekinator normally listens for inputs on port `6448`.

```
String oscAddress = "/wek/inputs";
```

This is the OSC message address that Wekinator expects for input data. Wekinator listens for feature vectors at `/wek/inputs`.

13.4. Local OSC Port

```
int localPort = 12001;
```

This is the local port used by this Processing sketch. Since this sketch mainly sends OSC data rather than receiving it, the exact local port is not very important. It simply allows oscP5 to start correctly.

13.5. Feature Configuration

```
int cropSize = 200;
```

This sets the crop size to 200×200 pixels. The sketch extracts a square region of this size from the webcam image.

```
int featureSize = 50;
```

This sets the downsampled feature image size to 50×50 pixels. After the 200×200 crop is taken, it is resized to 50×50.

```
boolean useRGB = false;
```

This chooses the color mode. If `useRGB` is `false`, the sketch sends grayscale brightness values. If it is `true`, it sends red, green, and blue values for each pixel.

```
int featureCount;
```

This variable will store the number of input values sent to Wekinator. It is calculated during `setup()`.

13.6. Image Variables

```
PImage cropImg;
```

This stores the 200×200 cropped image taken from the webcam.

```
PImage featureImg;
```

This stores the resized feature image, currently 50×50 pixels.

13.7. Preview and Sending Controls

```
boolean mirrorPreview = true;
```

This controls whether the camera preview is mirrored. A mirrored preview feels natural because it behaves like a mirror.

```
boolean sendToWekinator = true;
```

This controls whether the sketch is actively sending features to Wekinator. It can be toggled with the spacebar.

14. The `setup()` Function

The `setup()` function runs once when the sketch starts.

```
void setup() {
```

This starts the setup function.

```
size(960, 720);
```

This creates a Processing window with width 960 pixels and height 720 pixels.

14.1. OSC Setup

```
oscP5 = new OscP5(this, localPort);
```

This starts oscP5 on the local port. It prepares the sketch to send OSC messages.

```
wekinator = new NetAddress(wekHost, wekPort);
```

This creates the destination address for Wekinator. It tells Processing to send OSC messages to `127.0.0.1:6448`.

14.2. Camera Setup

```
String[] cameras = Capture.list();
```

This asks Processing to list all available camera devices.

```
if (cameras == null || cameras.length == 0) {
```

This checks whether no camera was found.

```
println("No camera found.");
```

This prints an error message in the console if there is no camera.

```
exit();
```

This stops the sketch because the program cannot work without a camera.

```
}
```

This closes the `if` block.

```
println("Available cameras:");
```

This prints a heading in the console before listing the available cameras.

```
for (int i = 0; i < cameras.length; i++) {
```

This starts a loop through all detected cameras.

```
println(i + ": " + cameras[i]);
```

This prints each camera with its index number.

```
}
```

This ends the loop.

```
cam = new Capture(this, cameras[0]);
```

This creates a webcam capture object using the first camera in the list.

```
cam.start();
```

This starts the webcam.

14.3. Feature Count Calculation

```
if (useRGB) {
```

This checks whether the sketch is using RGB mode.

```
featureCount = featureSize * featureSize * 3;
```

If RGB is used, every pixel creates three values: red, green, and blue. Therefore the number of features is width $\times$ height $\times$ 3.

```
} else {
```

This means if RGB mode is not used, the sketch uses grayscale mode.

```
featureCount = featureSize * featureSize;
```

In grayscale mode, every pixel creates one brightness value. Therefore the number of features is width $\times$ height.

```
}
```

This closes the feature count condition.

For the current configuration:

```
featureSize = 50
useRGB = false
featureCount = 50 x 50 = 2500
```

14.4. Console Information

```
println("=====================================");
```

This prints a separator line in the console.

```
println("Exercise 2 Input Sketch Started");
```

This confirms that the sketch has started.

```
println("Crop size: " + cropSize + " x " + cropSize);
```

This prints the crop size.

```
println("Feature image size: " + featureSize + " x " +
   featureSize);
```

This prints the downsampled image size.

```
println("Color mode: " + (useRGB ? "RGB" : "Grayscale"));
```

This prints either `RGB` or `Grayscale`, depending on the value of `useRGB`.

```
println("Number of Wekinator inputs: " + featureCount);
```

This prints the number of inputs that Wekinator must be configured to receive.

```
println("OSC address: " + oscAddress);
```

This prints the OSC address used for input messages.

```
println("Sending to: " + wekHost + ":" + wekPort);
```

This prints the destination host and port.

```
println("=====================================");
```

This prints another separator line.

14.5. Font Setup

```
textFont(createFont("Arial", 16));
```

This sets the font used for the text displayed in the Processing window.

```
}
```

This closes the `setup()` function.

15. The `captureEvent()` Function

```
void captureEvent(Capture c) {
```

This function is automatically called whenever a new camera frame is available.

```
c.read();
```

This reads the newest camera frame from the webcam.

```
}
```

This closes the function.

This function is important because without `c.read()`, the camera image would not update properly.

16. The `draw()` Function

The `draw()` function runs repeatedly. In Processing, it normally runs many times per second.

This is the real-time loop of the program.

```
void draw() {
```

This starts the draw loop.

```
background(0);
```

This clears the window with a black background.

```
if (cam.width == 0 || cam.height == 0) {
```

This checks whether the camera image is not ready yet.

```
return;
```

If the camera is not ready, the function stops for this frame and waits for the next frame.

```
}
```

This closes the camera readiness check.

```
cam.loadPixels();
```

This loads the camera pixels into memory so Processing can read and process them.

16.1. Crop the Camera Frame

```
cropImg = getCenterCrop(cam, cropSize);
```

This calls the `getCenterCrop()` function and stores the 200×200 cropped image in `cropImg`.

16.2. Downsample the Crop

```
featureImg = cropImg.copy();
```

This makes a copy of the cropped image. This is important because resizing should not destroy the original crop preview.

```
featureImg.resize(featureSize, featureSize);
```

This resizes the copied crop to the selected feature size, currently 50×50.

```
featureImg.loadPixels();
```

This loads the pixels of the resized image so they can be read and converted into feature values.

16.3. Send Features to Wekinator

```
if (sendToWekinator) {
```

This checks whether sending is currently enabled.

```
sendImageFeaturesToWekinator(featureImg);
```

This sends the current feature image to Wekinator as a list of numerical values.

```
}
```

This closes the sending condition.

16.4. Draw the Interface

```
drawCameraPreview();
```

This draws the full webcam preview and the green crop rectangle.

```
drawCropPreview();
```

This draws the 200×200 cropped image preview on the right side of the window.

```
drawFeaturePreview();
```

This draws the downsampled 50×50 feature image preview.

```
drawHUD();
```

This draws the information panel showing OSC settings, crop size, feature size, and sending status.

```
}
```

This closes the `draw()` function.

17. The `getCenterCrop()` Function

This function extracts a square region from the center of the webcam image.

```
PImage getCenterCrop(PImage source, int size) {
```

This defines a function that receives a source image and crop size, then returns a `PImage` crop.

```
int startX = (source.width - size) / 2;
```

This calculates the x-coordinate where the crop should start so that it is centered horizontally.

```
int startY = (source.height - size) / 2;
```

This calculates the y-coordinate where the crop should start so that it is centered vertically.

```
startX = constrain(startX, 0, source.width - size);
```

This prevents `startX` from going outside the valid image range.

```
startY = constrain(startY, 0, source.height - size);
```

This prevents `startY` from going outside the valid image range.

```
return source.get(startX, startY, size, size);
```

This extracts and returns the square crop from the source image.

```
}
```

This closes the function.

18. The `sendImageFeaturesToWekinator()` Function

This is one of the most important functions in the sketch. It converts an image into OSC feature values and sends them to Wekinator.

```
void sendImageFeaturesToWekinator(PImage img) {
```

This defines a function that receives an image and sends it to Wekinator as features.

```
OscMessage msg = new OscMessage(oscAddress);
```

This creates a new OSC message with the address `/wek/inputs`.

```
img.loadPixels();
```

This loads the image pixels so they can be accessed.

```
for (int i = 0; i < img.pixels.length; i++) {
```

This starts a loop through every pixel in the image.

For a 50×50 image:

```
img.pixels.length = 2500
```

```
color c = img.pixels[i];
```

This gets the current pixel color.

18.1. RGB Mode

```
if (useRGB) {
```

This checks whether RGB mode is enabled.

```
msg.add(red(c) / 255.0);
```

This extracts the red value from the pixel, normalizes it from 0–255 to 0–1, and adds it to the OSC message.

```
msg.add(green(c) / 255.0);
```

This extracts the green value, normalizes it, and adds it to the message.

```
msg.add(blue(c) / 255.0);
```

This extracts the blue value, normalizes it, and adds it to the message.

In RGB mode, each pixel adds three values.

18.2. Grayscale Mode

```
} else {
```

This runs when `useRGB` is false.

```
float gray = brightness(c) / 255.0;
```

This calculates the brightness of the pixel and normalizes it to the range 0–1.

```
msg.add(gray);
```

This adds the grayscale brightness value to the OSC message.

```
}
```

This closes the RGB/grayscale condition.

18.3. End of Pixel Loop and Send Message

```
}
```

This closes the pixel loop.

```
oscP5.send(msg, wekinator);
```

This sends the completed OSC message to Wekinator.

```
}
```

This closes the function.

19. The `drawCameraPreview()` Function

This function draws the full camera view and the green crop rectangle.

```
void drawCameraPreview() {
```

This starts the function.

```
pushMatrix();
```

This saves the current drawing transformation state.

```
if (mirrorPreview) {
```

This checks whether the preview should be mirrored.

```
translate(width, 0);
```

This moves the drawing origin to the right side of the window.

```
scale(-1, 1);
```

This flips the drawing horizontally.

```
}
```

This closes the mirror condition.

```
image(cam, 0, 0, width, height);
```

This draws the webcam image across the whole Processing window.

```
popMatrix();
```

This restores the drawing transformation so that later drawings are not mirrored.

19.1. Draw the Green Crop Rectangle

```
noFill();
```

This makes the rectangle empty inside.

```
stroke(0, 255, 0);
```

This sets the outline color to green.

```
strokeWeight(3);
```

This sets the rectangle border thickness to 3 pixels.

```
int rectX = (width - cropSize) / 2;
```

This calculates the x-position for the crop rectangle in the preview window.

```
int rectY = (height - cropSize) / 2;
```

This calculates the y-position for the crop rectangle in the preview window.

```
rect(rectX, rectY, cropSize, cropSize);
```

This draws the green square crop guide.

```
}
```

This closes the function.

20. The `drawCropPreview()` Function

This function draws a preview of the 200×200 cropped image.

```
void drawCropPreview() {
```

This starts the function.

```
int previewSize = 160;
```

This sets the display size of the crop preview to 160×160 pixels.

```
int x = width - previewSize - 20;
```

This places the preview near the right side of the window.

```
int y = 20;
```

This places the preview near the top of the window.

```
fill(0, 180);
```

This sets a semi-transparent black color.

```
noStroke();
```

This removes the outline from the background panel.

```
rect(x - 10, y - 10, previewSize + 20, previewSize + 45);
```

This draws a dark background box behind the crop preview.

```
image(cropImg, x, y, previewSize, previewSize);
```

This draws the cropped 200×200 image as a 160×160 preview.

```
fill(255);
```

This sets the text color to white.

```
textSize(14);
```

This sets the text size to 14.

```
text("Center crop: 200 x 200", x, y + previewSize + 25);
```

This labels the crop preview.

```
}
```

This closes the function.

21. The `drawFeaturePreview()` Function

This function draws the downsampled image that is actually used as Wekinator input.

```
void drawFeaturePreview() {
```

This starts the function.

```
int previewSize = 160;
```

This sets the preview size to 160×160 pixels.

```
int x = width - previewSize - 20;
```

This places the feature preview on the right side.

```
int y = 240;
```

This places the feature preview below the crop preview.

```
fill(0, 180);
```

This sets the panel color to semi-transparent black.

```
noStroke();
```

This removes the outline.

```
rect(x - 10, y - 10, previewSize + 20, previewSize + 65);
```

This draws a dark panel behind the feature preview.

21.1. Draw RGB Feature Preview

```
if (useRGB) {
```

This checks whether RGB mode is active.

```
image(featureImg, x, y, previewSize, previewSize);
```

If RGB mode is active, the feature image is displayed normally in color.

21.2. Draw Grayscale Feature Preview

```
} else {
```

This runs when grayscale mode is active.

```
PImage grayPreview = createImage(featureImg.width,
   featureImg.height, RGB);
```

This creates a new image for showing the grayscale preview.

```
featureImg.loadPixels();
```

This loads the feature image pixels.

```
grayPreview.loadPixels();
```

This loads the pixels of the new grayscale preview image.

```
for (int i = 0; i < featureImg.pixels.length; i++) {
```

This loops through all feature image pixels.

```
float gray = brightness(featureImg.pixels[i]);
```

This calculates the brightness of each pixel.

```
grayPreview.pixels[i] = color(gray);
```

This sets the grayscale preview pixel to that brightness value.

```
}
```

This closes the loop.

```
grayPreview.updatePixels();
```

This updates the preview image after modifying its pixels.

```
image(grayPreview, x, y, previewSize, previewSize);
```

This draws the grayscale preview image.

```
}
```

This closes the RGB/grayscale display condition.

21.3. Label the Feature Preview

```
fill(255);
```

This sets the text color to white.

```
textSize(14);
```

This sets the text size.

```
text("Feature image: " + featureSize + " x " + featureSize, x,
    y + previewSize + 25);
```

This displays the feature image size.

```
text("Mode: " + (useRGB ? "RGB" : "Grayscale"), x, y +
    previewSize + 45);
```

This displays whether the sketch is using RGB or grayscale mode.

```
}
```

This closes the function.

22. The `drawHUD()` Function

The HUD is the information panel shown on the screen.

```
void drawHUD() {
```

This starts the HUD drawing function.

```
fill(0, 190);
```

This sets a semi-transparent black fill.

```
noStroke();
```

This removes the outline.

```
rect(20, 20, 390, 170);
```

This draws the dark background panel.

```
fill(255);
```

This sets text color to white.

```
textSize(16);
```

This sets the title text size.

```
text("Exercise 2 -- Gesture Input Sketch", 35, 50);
```

This displays the sketch title.

22.1. OSC and Feature Information

```
textSize(14);
```

This sets smaller text for the information lines.

```
text("OSC address: " + oscAddress, 35, 80);
```

This displays the OSC message address.

```
text("Sending to Wekinator: " + wekHost + ":" + wekPort, 35,
    105);
```

This displays the Wekinator destination host and port.

```
text("Crop: " + cropSize + " x " + cropSize, 35, 130);
```

This displays the crop size.

```
text("Feature size: " + featureSize + " x " + featureSize, 35,
    155);
```

This displays the resized feature image size.

```
text("Color mode: " + (useRGB ? "RGB" : "Grayscale"), 35, 180);
```

This displays the current color mode.

```
text("Wekinator inputs: " + featureCount, 35, 205);
```

This displays the number of Wekinator inputs required.

22.2. Sending Status

```
fill(sendToWekinator ? color(0, 255, 0) : color(255, 0, 0));
```

This chooses the text color based on whether sending is active. Green means sending is ON. Red means sending is OFF.

```
text("Sending: " + (sendToWekinator ? "ON" : "OFF"), 35, 230);
```

This displays the sending state.

```
fill(255);
```

This changes text color back to white.

```
text("Press SPACE to start/stop sending", 35, height - 35);
```

This tells the user that the spacebar toggles OSC sending.

```
}
```

This closes the HUD function.

23. The `keyPressed()` Function

This function handles keyboard input.

```
void keyPressed() {
```

This function runs whenever the user presses a key.

```
if (key == ' ') {
```

This checks whether the key pressed is the spacebar.

```
sendToWekinator = !sendToWekinator;
```

This toggles the sending state. If sending was ON, it becomes OFF. If it was OFF, it becomes ON.

```
println("Sending to Wekinator: " + sendToWekinator);
```

This prints the new sending state in the console.

```
}
```

This closes the `if` statement.

```
}
```

This closes the `keyPressed()` function.

24. Complete Program Flow

The full execution flow is:

```
1.  Processing starts the sketch.
2.  setup() runs once.
3.  OSC communication is prepared.
4.  The webcam is detected and started.
5.  featureCount is calculated.
6.  draw() starts repeating.
7.  The newest camera frame is read.
8.  The center 200x200 image region is cropped.
9.  The crop is resized to 50x50.
10. Each pixel is converted into a grayscale value.
11. 2500 values are added to an OSC message.
12. The OSC message is sent to Wekinator.
```

```
13. The camera preview, crop preview, feature preview, and HUD
   are drawn.
14. The process repeats continuously in real time.
```

25. Wekinator Training Procedure with This Sketch

Step 1 — Run the Input Sketch

Start `GestureInput.pde` in Processing.

Make sure the console shows:

```
Number of Wekinator inputs: 2500
OSC address: /wek/inputs
Sending to: 127.0.0.1:6448
```

Step 2 — Configure Wekinator

Use these project settings:

```
Input port: 6448
Input OSC message: /wek/inputs
Number of inputs: 2500
Output OSC message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 1
Output type: Classification
Number of classes: 2
```

Step 3 — Choose Two Gestures

Recommended first experiment:

```
Class 1: Open Hand
Class 2: Closed Fist
```

Step 4 — Record Class 1

Put the Class 1 gesture inside the green crop box. Start recording in Wekinator. Move the hand slightly while keeping the same gesture. Stop recording.

Step 5 — Record Class 2

Put the Class 2 gesture inside the same crop box. Start recording. Move the hand slightly. Stop recording.

Step 6 — Train the Classifier

Use KNN with:

```
k = 3
```

Then click **Train**.

Step 7 — Run the Model

Click **Run** in Wekinator.

Step 8 — Run the Output Sketch

The output sketch should listen on port `12000` and display the predicted gesture class.

26. Important Training Advice

The most important rule is:

```
The gesture must be inside the green crop box during both
   training and testing.
```

If the crop contains mostly your face or background, Wekinator will learn the wrong information.

Good training examples should include:

- Hand clearly inside the crop area
- Similar lighting during training and testing
- Simple background
- Balanced number of examples for each class
- Slight variations in hand position
- Same approximate camera distance

Bad training examples include:

- Hand outside the crop box
- Face inside the crop box instead of hand
- Too much background
- Big lighting changes
- Recording Class 1 while accidentally showing Class 2
- Very unbalanced examples between classes

27. Why KNN Works with This Input

Wekinator receives a 2500-dimensional vector from each frame.

Each recorded example has:

```
2500 input values + class label
```

For example:

```
Example 1: 2500 grayscale values -> Class 1
Example 2: 2500 grayscale values -> Class 1
Example 3: 2500 grayscale values -> Class 2
```

When a new camera frame arrives, KNN compares the new 2500-value vector with the stored training examples.

With `k = 3`, it looks at the three nearest examples and uses majority vote.

Example:

```
Nearest example 1 -> Class 1
Nearest example 2 -> Class 1
Nearest example 3 -> Class 2
Final prediction  -> Class 1
```

This is why balanced and clear training examples are important.

28. How This Code Fulfils the Exercise 2 Requirements

Exercise 2 requires a system for real-time gesture classification using video input and Wekinator.

This sketch fulfils the input part of the project as follows:

Exercise Requirement	How This Code Fulfils It
Capture real-time video input from webcam	Uses `Capture cam` from Processing Video library
Crop central region of image	Uses `getCenterCrop(cam, cropSize)` with `cropSize = 200`
Use 200×200 crop	`int cropSize = 200`
Generate downsampled versions	Uses `featureImg.resize(featureSize, featureSize)`
Support 50×50 input	Uses `int featureSize = 50`
Support grayscale representation	Uses `boolean useRGB = false` and `brightness(c)`
Support RGB representation	If `useRGB = true`, sends red, green, and blue values
Convert pixels into feature vector	Loops through `img.pixels` and adds values to OSC message
Send feature vector to Wekinator	Uses `OscMessage` and `oscP5.send(msg, wekinator)`
Use OSC communication	Uses oscP5 and netP5 libraries

Exercise Requirement	How This Code Fulfils It
Match Wekinator input message	Sends to `/wek/inputs`
Match Wekinator input port	Sends to port `6448`
Provide real-time operation	Runs continuously inside `draw()`
Allow debugging/monitoring	Displays crop preview, feature preview, HUD, and console logs

29. Strengths of This Sketch

This sketch is useful for the project because it has the following strengths:

29.1. Clear Pipeline

The processing pipeline is clear and modular:

```
Camera -> Crop -> Resize -> Feature extraction -> OSC send
```

29.2. Easy Feature Configuration

The user can test different feature modes by changing:

```
int featureSize = 50;
boolean useRGB = false;
```

29.3. Good First Configuration

The default setting, 50×50 grayscale, is a good starting point because it is lighter than the full 200×200 image and less complex than RGB.

29.4. Real-Time Visual Feedback

The sketch shows:

- Full camera preview
- Green crop box
- Cropped image preview
- Downsampled feature image preview
- Number of Wekinator inputs
- OSC address and port
- Sending ON/OFF status

This makes debugging much easier.

29.5. Spacebar Control

The user can stop sending data by pressing the spacebar. This is useful when setting up Wekinator or adjusting the camera.

29.6. Wekinator Compatibility

The OSC settings match the common Wekinator workflow:

```
Processing input sketch -> /wek/inputs -> Wekinator
```

30. Limitations of This Sketch

The sketch works well as an input sketch, but it has some limitations.

30.1. Center Crop May Not Always Capture the Hand

The crop is fixed in the center. If the hand is on the side of the image, the crop may capture the face instead of the gesture.

Possible improvement:

```
Move the crop box manually using keyboard controls or mouse.
```

30.2. Background Can Affect Classification

Since the feature vector is based on raw pixels, the classifier can be affected by background objects and lighting.

Possible improvement:

```
Use a plain background and consistent lighting.
```

30.3. Grayscale Removes Color Information

Grayscale is simpler, but it loses color information.

This is acceptable for shape-based gestures but may be less useful if color is important.

30.4. Pixel Features Are High-Dimensional

Even the 50×50 grayscale version sends 2500 values. Larger modes can become very heavy.

For example:

```
200x200 RGB = 120,000 inputs
```

This can be difficult for Wekinator to process.

31. Recommended Experimental Results Discussion

For the final lab report, the following discussion can be included.

31.1. Resolution Comparison

Resolution	Expected Behavior
200×200	Preserves most detail but produces many inputs and may be slow/heavy
100×100	Good balance between detail and complexity
50×50	Fast and simple; enough for clear gestures like open hand vs fist

31.2. Grayscale vs RGB Comparison

Mode	Expected Behavior
Grayscale	Lower dimensionality, focuses on shape and brightness
RGB	More information, but more inputs and more sensitivity to lighting

For simple hand gesture classification, grayscale can be enough because the difference between an open hand and closed fist is mainly shape-based.

32. Debugging Checklist

If Wekinator is not receiving data:

- Check that the input sketch is running.
- Check that `sendToWekinator` is ON.
- Check that Wekinator is listening on port `6448`.
- Check that Wekinator input message is `/wek/inputs`.
- Check that Wekinator input count is `2500`.
- Check that no other program is blocking OSC communication.

If classification is wrong:

- Make sure the hand is inside the green crop box.
- Delete bad examples and retrain.
- Record balanced examples for both classes.
- Use consistent lighting.
- Keep the same camera distance.
- Set KNN to `k = 3`.

- Train again after changing settings.

If the output sketch does not change:

- Make sure Wekinator is in Run mode.
- Make sure the output sketch is listening on port `12000`.
- Make sure Wekinator sends to `/wek/outputs`.
- Make sure the output type is classification with one output and two classes.

33. Final Explanation in Simple Words

This sketch turns a camera image into numbers for Wekinator.

It does not know what a hand is. It does not know what a gesture is. It simply extracts a square part of the camera image, makes it smaller, converts the pixels into numerical brightness values, and sends those values to Wekinator.

Wekinator then learns patterns in those numbers.

If the open hand always creates one kind of pixel pattern and the closed fist creates another kind of pixel pattern, Wekinator can learn to classify them.

The most important practical point is that the gesture must be clearly visible inside the crop area.

34. Final Conclusion

The `GestureInput.pde` sketch is a correct and useful input sketch for Exercise 2.

It fulfils the main requirements of the gesture classification exercise by capturing webcam video, cropping a 200×200 region, downsampling it to a selected resolution, converting pixel data into feature vectors, and sending those features to Wekinator through OSC.

The current configuration uses:

```
200x200 crop
50x50 feature image
Grayscale pixel values
2500 Wekinator inputs
OSC message /wek/inputs
OSC port 6448
```

This is a good first configuration because it is lighter and easier to train than larger RGB configurations.

The sketch also includes useful debugging tools, such as the green crop box, crop preview, feature preview, HUD information, and the ability to pause sending with the spacebar.

In the full project, this sketch forms the first half of the interactive machine learning pipeline:

```
Gesture shown to webcam
        |
Processing input sketch extracts features
        |
Wekinator classifies the gesture using KNN
        |
Processing output sketch displays the recognized gesture
```

Therefore, this code successfully supports the core goal of the lab: building an interactive real-time gesture classification system using Processing, OSC, and Wekinator.